# Detecting Silent Implementation Integrity Failures in AI-Generated Code: A Static Analysis Approach

Avnish Singh
Babu Banarasi Das University, Lucknow, India
avnishsingh150606@gmail.com

## Abstract

When an AI coding agent reports that it has finished wiring a safety check into an execution path, the resulting code may parse, import, and pass its existing tests while the check itself never runs. We call this a *silent implementation integrity failure*: the structure of the code is present, the function that performs the work is not connected to anything, and no error is raised. This paper describes the Implementation Integrity Analyzer (IIA), a deterministic static analyzer that looks for three signatures of this failure in Python source — functions whose bodies contain only recognized scaffold markers, stated intent that does not correspond to any identifier in the implementation, and safety-critical functions that are defined but never invoked from the operations they are meant to guard. We evaluate IIA on a labeled benchmark of fifteen constructed scenarios against a string-presence baseline. IIA reaches MCC 0.389 (precision 0.556, recall 0.833). The baseline degenerates to a single-class predictor — a function name always occurs in its own definition, so a presence check cannot separate a gate that exists from a gate that runs — leaving its precision and MCC mathematically undefined. All fifteen scenarios are accounted for: the analyzer classifies the ten straightforward cases correctly and produces four false positives and one false negative on the five cases constructed to sit outside its documented analysis boundary. We report these errors without adjustment and describe what each one reveals about the limits of intra-file AST analysis. The benchmark is small and self-authored; the results characterize one detector on one corpus and do not estimate real-world prevalence or deployment performance.

**Keywords:** coding agents, static analysis, AST analysis, software verification, silent failures, code review tooling

## 1 Introduction

Something that happens with AI coding agents, and that is awkward to catch, is this: the agent writes a function that is supposed to guard an operation, writes the operation, reports that both are done, and never connects them.

We ran into this while building AegisOps, a crisis decision-support system in which a deterministic rule layer is supposed to gate every downstream action. The agent was asked to wire the validation gate into the decision engine. It produced the gate with a full signature and a docstring describing exactly what the gate was supposed to prevent, created the test scaffolding around it, and reported the task complete. The gate was never called. The decision path executed without it. This happened in two separate sessions before we caught it during manual review, and in neither case did anything fail loudly enough to notice.

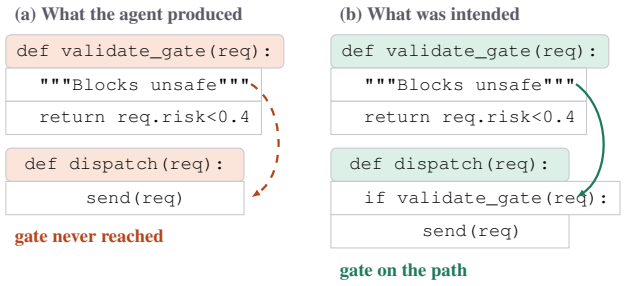


**Figure 1.** Both files parse, import, and expose the same public surface. In (a) the safety gate is defined but never appears on the dispatch path, so the guarded operation executes unconditionally. No exception is raised and no test fails unless a test happens to exercise the guarded path end to end.

We should be precise about the epistemic status of that story: it is a development observation from a single project, recorded informally, not a controlled experiment. It motivated this work; it does not by itself establish a frequency or a base rate. The empirical content of this paper is the benchmark in Section 4, and that benchmark is small.

What makes this class of failure awkward is that nothing about it looks like an error. The file parses. The imports resolve. If the existing tests do not happen to exercise the guarded path end to end — and tests written by the same agent in the same session frequently do not — then the suite stays green. A crash announces itself. An unwired gate does not announce anything at all, and the system behaves correctly right up until the moment the guard was supposed to matter. Figure 1 shows the shape of the problem.

Recent empirical work has begun to document adjacent phenomena. Mehta [1] measures the gap between agents submitting patches and those patches being verified correct, finding that GPT-5 submits on every SWE-bench Verified run while resolving 44% of them, and names the dominant mode *silent semantic failure* — confidently and repeatedly wrong in a way that completion-based monitoring cannot see. Parris [2] audits AI-attributed source against matched human controls and reports a directional skew toward code that fails quietly, concentrated in exception-handling patterns. Al Hasan and

Biswas [3] mine GitHub issues from deployed coding tools and confirm 547 genuine safety failures, of which 122 involve agents taking destructive shortcuts to make errors disappear rather than resolving them.

These studies establish that the phenomenon is real and measurable at the level of trajectories, incidents, and file-level statistics. What they do not provide is a detector that examines a single file and reports which specific structural connection is missing. That is the gap this paper addresses, in a deliberately narrow way.

**Contributions.** (i) We name and characterize three structural signatures of silent integrity failure that are checkable from a Python AST alone. (ii) We implement IIA, a static analyzer with no external dependencies, no learned components, and no runtime instrumentation, so that its output is reproducible and every flag traces to a specific syntactic observation. (iii) We evaluate against a fifteen-scenario labeled corpus with a string-presence baseline, report all errors including the unflattering ones, and connect each error to a limitation documented in the implementation before evaluation was run.

## 2 Related Work

**Silent failure in agent behavior.** SWE-bench [8] fixed the task format most of this literature now assumes: hand an agent a repository and an issue, let it edit, and score the edit by whether the project's tests pass. Mehta [1] pulls apart two quantities that format tends to conflate — how often an agent submits something, and how often what it submits survives testing — over 1,750 trajectories on 50 SWE-bench Verified tasks, and finds the gap between them is systematic. We take something narrower from this than the paper argues for. If a completion signal cannot be trusted, the artifact behind it has to be looked at directly, and that is the level IIA works at.

**Failure-untruthful code.** Of the prior work we know of, AIRA [2] is closest in spirit. It proposes that training on human feedback may push generation toward code that looks right rather than code that fails loudly — the Reward-Shaped Failure Hypothesis — and tests this with a fifteen-check deterministic inspection framework run over 955 AI-attributed files and an equal number of human-written controls, reporting 0.435 versus 0.242 high-severity findings per file. The overlap with our work is the method, not the target. AIRA is mostly hunting exception handlers that absorb failure signals; we are hunting safety logic that exists but nothing routes through.

**Incident taxonomies.** Three recent efforts catalogue what agents get wrong. Al Hasan and Biswas [3] work from mined GitHub issues — 16,586 of them, filtered from an initial screen of 68,816 papers — and end up with 33 operational risk types, 326 of 547 confirmed incidents landing in the high or critical band. Ge et al. [4] sort failures instead by where they originate: underspecified tasks, limits of the model, or the harness around it, tested across 20 coding environments and 59 synthetic transcript templates. Pathak et al. [5] move the question to multi-agent settings and treat it as anomaly detection over execution traces, with labelled trajectory sets of 4,275 and 894. These tell us what breaks and roughly how often. None of them hands a reviewer a check to run against a file.

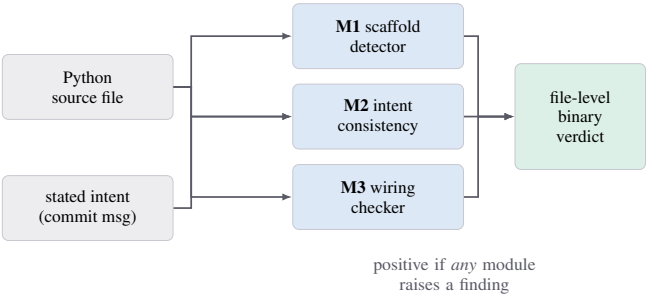


**Figure 2.** The three detector modules run independently over one file. For the headline evaluation their outputs are collapsed into a single binary verdict per scenario.

**Static analysis and the false-positive tax.** The constraints IIA operates under are old ones. PyCG [9] is the reference point for what a serious Python call graph costs: an inter-procedural analysis resolving assignments between functions, variables, classes and modules, reaching roughly 99.2% precision and 69.9% recall. Everything we exclude in Section 6 — aliases, transitive chains, cross-module edges — is machinery PyCG has and we do not. Whether that machinery is worth its complexity for a pre-merge check is a separate question, and one we have not answered. Adoption research makes the stakes concrete on the other side: Johnson et al. [12] interviewed twenty developers and found false positives and poor warning presentation to be primary reasons static analysis tools go unused, and Christakis and Bird [13] report the same pattern from a different population. A detector that over-flags is not merely imprecise; it gets switched off.

**Agent-side tooling.** A second family of tools constrains the agent rather than inspecting what it produced. Ponytail [6] injects behavioural rules into a coding session to hold down over-generation. Goose [7], now under the Linux Foundation's Agentic AI Foundation, ships an egress inspector for outbound network calls. Neither looks at the resulting source, which is all IIA looks at, so the two approaches sit beside each other rather than compete.

**Positioning.** The work above measures failure rates, sorts failures into categories, or intervenes on agent behaviour. IIA does none of these. It asks one question of one file — is this safety function reachable from the operation it guards — and answers it in a form a reviewer can check by reading the code.

## 3 Method

IIA analyzes Python source using the standard library `ast` module. There is no model, no embedding, no network call during analysis, and no execution of the code under inspection. The same input produces the same output. Figure 2 shows how the three modules combine.

### 3.1 Scaffolded function detection

The first module walks `ast.FunctionDef` and `ast.AsyncFunctionDef` nodes and classifies each body as scaffolded or implemented. A body counts as scaffolded when it contains only recognized scaffold markers: `pass`, an ellipsis literal, a docstring with no following statements, or `raise NotImplementedError(...)` as its sole statement. Anything else is treated as implemented.

The detector operates on the parsed AST, which does not retain comments. It therefore cannot distinguish a body that is empty from one that is empty apart from comments, and we make no claim that it detects comment-only bodies as a separate category.

The signal being exploited is the mismatch between what a signature and docstring advertise and what the body contains. A function named `validate_safety_gate` whose docstring describes the conditions it rejects, and whose body is `pass`, is a candidate for review regardless of why it ended up that way. The check does not attempt to judge whether an implemented body is *correct*; it only distinguishes scaffold markers from substance, which is a much weaker claim than the module name might suggest.

### 3.2 Intent–implementation consistency

The second module compares a short natural-language intent string — in practice a commit message or task description — against the identifiers that appear in the source.

Capability keywords are extracted from the intent by whitespace tokenization, stopword removal, and a minimum-length filter. Identifiers are collected from the source via `ast.walk`: function definitions, called names, imported names, and assigned variables. Each keyword is matched against the identifier set using `difflib.get_close_matches`. Keywords with no close match are reported as claimed-but-absent, and the module returns a consistency score of matched over total.

We want to be direct about what this is. It is fuzzy string matching between two bags of tokens. It is not natural-language understanding: it does not model meaning, negation, or scope. It catches the case where an intent mentions retry logic and the file contains no identifier resembling retry; it will not catch an intent satisfied by differently-named code, and it will not catch code that uses the right names to do the wrong thing. "Plan-vs-execution" is the role the check plays in the pipeline, not a description of its mechanism.

The benchmark harness converts the continuous consistency score into a binary finding using a threshold of **0.4**. This value is a harness convention chosen to make the module comparable with the other two detectors. It is not derived from theory, is not claimed to be optimal, and was not tuned against the results reported here.

### 3.3 Safety-critical wiring

The third module is the one the paper is really about.

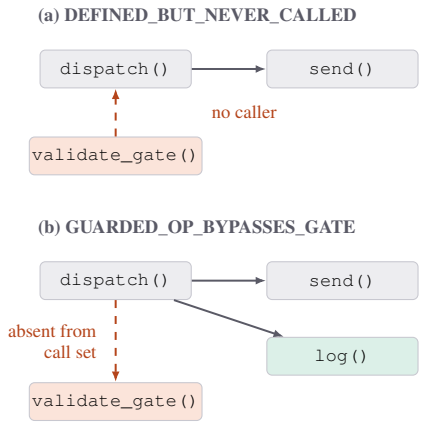


**Figure 3.** The two conditions reported by the wiring checker, shown as call maps. Dashed red edges mark the connection the checker expects and does not find. Both are membership questions over the collected call sets, not reachability queries.

It builds an intra-file call map by traversing each function definition and collecting the names appearing in `ast.Call` nodes within that subtree. Two configurable name sets drive the analysis: safety-critical function names (for example `validate_safety_gate`, `requires_human_approval`, `check_constraints`) and operation keywords marking work that ought to be guarded (`dispatch`, `allocate`, `execute`, `commit`).

Because collection walks the whole function subtree, calls appearing inside nested function definitions are attributed to the enclosing function's call set. We note this as a property of the current representation rather than a designed behavior.

Two conditions are then reported, illustrated in Figure 3:

**DEFINED_BUT_NEVER_CALLED.** A safety-critical function is defined in the file but its name does not appear in any other function's call set. Self-recursion does not count; a gate that only calls itself is not wired into anything.

**GUARDED_OP_BYPASSES_GATE.** A function whose own name or whose called names match an operation keyword has a call set containing no safety-critical name at all.

Both conditions are set-membership questions over a call map built from one file. That framing is what makes the analysis cheap and deterministic, and it is also the source of every limitation in Section 6.

## 4 Evaluation

### 4.1 File-level scoring

Each scenario may produce findings from more than one module. For the headline confusion matrix, detector outputs are collapsed to a single file-level binary verdict: a scenario is classified positive if any module raises a finding, and clean otherwise. Individual finding categories are retained for the per-scenario analysis in Section 4.5 but are not scored as three separate binary tasks.

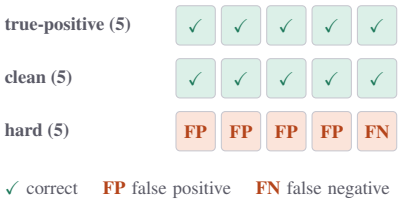


**Figure 4.** Per-scenario outcome for IIA across the three benchmark groups. Every error is confined to the hard group.

### 4.2 Corpus

The benchmark is fifteen self-contained Python files with ground-truth labels in `labels.json`, arranged in three groups of five (Figure 4).

Five **true-positive** scenarios each carry exactly one seeded defect: a scaffolded safety gate, a gate that is never called, a guarded operation that bypasses its gate, an intent–implementation mismatch, and a `NotImplementedError` stub in code that claims completion.

Five **clean** scenarios contain correctly wired gates, fully implemented functions, multiple gates all properly invoked, code with no safety-critical surface at all, and an operation calling its gate alongside additional checks. These exist to measure false alarms, which for a review tool matter as much as detections.

Five **hard** scenarios were written to sit at or beyond the documented boundary of the analysis: a gate reached through a variable alias, a gate reached through an intermediate helper, a legitimately empty abstract method, a decorator-based guard, and a gate that is called but inside the wrong conditional branch. Four are labeled clean — the safety property genuinely holds, so flagging them is an error. One, the wrong-branch case, is labeled positive, because the gate does not actually protect the operation.

Fifteen scenarios, all written by the author of the detector, is enough to characterize behavior on named, deliberately-chosen cases and not enough to estimate how often those cases occur in real repositories.

### 4.3 Baseline

The baseline is the check a developer performs first when asking whether a gate is used: search the file text for the gate's name.

The decision rule matters, and we state it explicitly because the direction is easy to misread. This baseline is a **presence/absence string check, not a call detector**. A textual hit is taken as evidence that the gate is present, so the baseline returns no finding and the scenario is predicted **clean**. Only the complete absence of the configured name from the source text would cause it to predict an integrity violation. It makes no attempt to distinguish a definition from a call, and it has no mechanism for detecting scaffolded bodies or intent mismatch.

The consequence is immediate and is the reason we include this baseline at all. A safety-critical function's name necessarily appears in its own `def` line, so the string is found in every file that defines a gate. The baseline therefore predicts clean on every scenario in the corpus and never predicts a positive. It is a degenerate, single-class predictor.

We note the cost of that degeneracy. A predictor that emits one class has $TP = FP = 0$, which makes precision $(0/0)$ and MCC (a denominator containing $TP+FP=0$) undefined rather than zero. Reporting 0.000 for MCC in Table 1 follows the convention used by standard implementations, including scikit-learn, of returning zero when the denominator vanishes; it is a convention, not a computed correlation. The baseline column should be read as "carries no information about the property being measured," not as "scored badly on a scale where IIA scored better."

This limits what the comparison establishes. It demonstrates that string presence is not a usable signal for wiring — a real and non-obvious point, since a name search is the reflexive first check — but it does not position IIA against a competent alternative. A stronger baseline would search for the gate name while excluding its own definition line, which would at least be capable of predicting both classes. We have not run that variant and claim no results for it.

### 4.4 Results

Both systems were run over all fifteen scenarios via `run_benchmark.py`, which writes per-scenario predictions to `results.json`.

| Metric | Baseline | IIA |
|---|---|---|
| True positives | 0 | **5** |
| False positives | 0 | 4 |
| False negatives | 6 | 1 |
| True negatives | 9 | **5** |
| Precision | undefined $(0/0)$ | 0.556 |
| Recall | 0.000 | **0.833** |
| MCC | undefined; 0.000$^\dagger$ | **0.389** |

**Table 1.** Confusion matrices and metrics over the fifteen-scenario corpus. $^\dagger$The baseline predicts a single class, so $TP+FP=0$ and the MCC denominator is zero; 0.000 is the conventional degenerate-case return, not a computed correlation. Recall is well defined at 0.000 because its denominator, $TP+FN$, is 6.

Matthews correlation coefficient is the headline number for IIA. With fifteen scenarios and six positives, accuracy would be dominated by the majority class and $F_1$ would ignore true negatives, which is exactly the quantity a review tool must not sacrifice. Chicco and Jurman [10] make the general case for preferring MCC on imbalanced data; Yao and Shepperd [11] make it specifically for software defect prediction, where the class ratios look much like ours.

### 4.5 What the numbers mean

The baseline's zero recall is structural rather than accidental. Every safety-critical function name appears in its own `def` line, so the search returns a hit for every file in which the

gate is defined, whether or not anything calls it. Under the decision rule of Section 4.3, a hit means clean, so the baseline predicts clean fifteen times out of fifteen and misses all six true positives.

We are not claiming this baseline is a serious competitor that IIA outperformed. It demonstrates that the reflexive first check carries no information about wiring, and that the failure is structural rather than a matter of tuning: no threshold adjustment recovers a signal from a string guaranteed to be present.

IIA is correct on all ten scenarios in the true-positive and clean groups. Every error falls in the hard group, and each maps to a limitation written into the module docstrings before the benchmark was run.

**The four false positives.**

*Alias.* The gate is assigned to a variable and invoked through it. The call map records the variable name, not the gate, so the gate appears uncalled.

*Helper indirection.* The operation calls a helper; the helper calls the gate. The analysis does not resolve transitive chains, so the operation's call set contains no safety-critical name.

*Abstract method.* A method whose body is a single `raise NotImplementedError()` is classified as scaffolded. The `@abstractmethod` decorator makes this the correct and intended implementation, and the scaffold detector does not read decorators.

*Decorator guard.* The safety check is applied by a decorator rather than called in the body. The wiring checker inspects call sets, not decorator lists.

**The single false negative** deserves its own paragraph, because it is the more interesting error. In `hard_gate_called_inside_wrong_branch.py`, the gate *is* called — it simply sits in a branch that does not execute before the guarded operation. The checker asks whether the gate name appears in the operation's call set, and it does, so the file is reported clean. The safety property does not hold, and the analyzer says nothing.

> This is the ordering limitation made concrete. **Presence in a call set is not equivalent to execution before the guarded action**, and no amount of threshold tuning fixes that within a set-membership model. It is the one case where the tool fails in the direction that matters most.

Nothing was excluded, relabeled, or reweighted after seeing these results.

### 4.6 Error direction

Four of the five errors are false positives. For a pre-merge review aid, this asymmetry is preferable in principle: a false positive generally requires additional human review, whereas a false negative can allow an unguarded operation to pass the check. We designed for that asymmetry, and on this corpus we got it — but with a single false negative out of six positives, the corpus is far too small to claim the bias is reliable. It is a design intent that these results are consistent with, not a demonstrated property. The cost of the false-positive side is also not zero: over-flagging is one of the documented reasons developers abandon static analysis tools [12, 13].

## 5 Reproduction

Results were produced under Python 3.13.5 with pytest 8.4.2, on the repository checkout that produced the numbers in Table 1.

```
python -m aegisops.integrity_analyzer\
       .benchmark.run_benchmark
```

Scenarios and labels live in `aegisops/integrity_analyzer/benchmark/scenarios/`; the run writes per-scenario predictions to `benchmark/results.json`. The repository's test suite passes 121 tests.

The implementation, benchmark corpus, ground-truth labels, baseline, and evaluation script are available at https://github.com/Avnish1505/aegisops-ai. We describe the materials as available rather than as a turnkey reproduction: the numbers above regenerate from that checkout with the command shown, and readers who find otherwise are asked to open an issue.

## 6 Limitations

**Intra-file only.** A gate defined in one module and called from another is invisible to the analysis, a substantial restriction given that this is how such code is usually organized. Repository-scale application would require cross-module call resolution the implementation does not have.

**No alias resolution.** The call map records names, not values. Assigning a gate to a variable and calling through it defeats the check.

**No transitive resolution.** Safety checks invoked indirectly through helper functions are not recognized as wired. PyCG [9] resolves exactly this class of edge, and a version of IIA built on it rather than on a local call map would not produce the helper-indirection false positive we report.

**No control-flow or ordering analysis.** Call-set membership carries no ordering information. The wrong-branch false negative in Section 4.5 is this limitation observed directly rather than merely asserted.

**No decorator semantics.** Decorator-based guards are not recognized as safety checks, and `@abstractmethod` is not recognized as making an empty body correct.

**Name collisions are not disambiguated.** The call map is keyed by function name rather than fully-qualified symbol identity, so same-named functions in different classes can overwrite one another in the current representation. This is a documented property of the implementation; no benchmark scenario tests it.

**Nested-function attribution.** Calls inside nested function definitions are attributed to the enclosing function's call set, which may mask or manufacture a wiring relationship.

**Manual configuration.** IIA does not discover which functions are safety-critical; someone must say so, and a wrong or incomplete configuration produces confident wrong answers.

**Evaluation scope.** Fifteen scenarios, written by the same person who wrote the detector, cannot establish generalization. The hard cases are constructed rather than mined from real repositories. Measuring how often these patterns occur in practice would require a corpus of real AI-generated commits with known ground truth, which we do not have.

## 7 Discussion

### 7.1 Where this fits in a workflow

IIA is a pre-merge review aid and nothing more. It does not replace tests, does not prove correctness, and makes no claim about semantic behavior; it flags structural patterns for human adjudication. Its narrow value is in surfacing a case that a passing test suite does not distinguish from success — particularly when the tests were written in the same session, by the same agent, as the code they exercise. Running it over a diff provides a lightweight structural check before review.

We are wary of stating this as "standard CI cannot detect this." A sufficiently thorough integration test that exercises the guarded path end to end would catch every true positive in our corpus. The claim is narrower and more defensible: unwired safety logic does not announce itself, and whether it is caught depends on whether some test happens to traverse that path. Static structural analysis does not depend on that contingency.

### 7.2 Why not a learned detector

A sufficiently capable learned code model could potentially handle aliasing and helper indirection better than the current call-map approach, but we did not evaluate such a detector. We chose deterministic analysis for three reasons that we think outweigh that.

*Reproducibility:* identical input yields identical output, with no sampling variance and no version drift in a hosted model.

*Auditability:* every flag points at a specific AST node, so a developer can confirm or dismiss it by reading the code rather than by trusting a score.

*A circularity concern,* which is more of a design intuition than an argument we can defend empirically: using a model with the same training pressures that produce the failure to detect the failure seems like a poor foundation for a check meant to be trusted. We note this as a motivation for our choice, not as a finding.

### 7.3 Generality

The pattern — structure present, connection absent — is not specific to Python or to code. Infrastructure definitions, policy configurations, and permission models can all be generated in a state where the guard exists and nothing routes through it. Whether the detection approach transfers is an open question we have not tested.

## 8 Conclusion

We described a class of failure in AI-generated code where safety logic is written but never connected to the operation it protects, implemented a deterministic static analyzer that checks for three structural signatures of it, and evaluated the analyzer against a string-presence baseline on a fifteen-scenario labeled corpus. IIA reaches MCC 0.389, recall 0.833, and precision 0.556. The baseline collapses to a single-class predictor whose precision and MCC are undefined, which establishes that textual presence carries no information about wiring but does not amount to a contest IIA won. All ten straightforward scenarios are classified correctly; the four false positives and one false negative fall entirely in the hard group and correspond to limitations documented before evaluation.

The corpus is small and constructed, and the result is a characterization of one detector on that corpus rather than an estimate of deployment performance. We report it in full, including the false negative that demonstrates the ordering limitation, because a modest result another researcher can reproduce and disagree with seems more useful than a stronger number on a corpus tuned until it cooperates.

## References


[1] A. Mehta. Confident and Wrong: Silent Semantic Failures in Coding Agents. Snowflake AI Research. arXiv:2603.25764.

[2] W. Parris. AIRA: AI-Induced Risk Audit: A Structured Inspection Framework for AI-Generated Code, 2026. arXiv:2604.17587.

[3] A. Al Hasan and S. Biswas. What Breaks When LLMs Code? Characterizing Operational Safety Failures of Agentic Code Assistants. Case Western Reserve University, 2026. arXiv:2605.30777.

[4] K. Ge et al. ClayBuddy: A Framework, Evaluation, & Mitigation of Coding Agent Failures, 2026. arXiv:2606.19380.

[5] D. Pathak, F. George, H. Kumar, A. Roy, K. Ray, M. Verma, and P. Moogi. Detecting Silent Failures in Multi-Agentic AI Trajectories. IBM Research. arXiv:2511.04032. Work-in-progress version in *ICPE Companion '26*, Florence, Italy, pp. 33–39. DOI: 10.1145/3777911.3801104.

[6] D. Gebert. Ponytail. https://github.com/DietrichGebert/ponytail

[7] Block. Goose. https://github.com/block/goose

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024. arXiv:2310.06770.

[9] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos. PyCG: Practical Call Graph Generation in Python. *ICSE*, 2021, pp. 1646–1657. DOI: 10.1109/ICSE43902.2021.00146.

[10] D. Chicco and G. Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. DOI: 10.1186/s12864-019-6413-7.

[11] J. Yao and M. Shepperd. Assessing software defection prediction performance: Why using the Matthews correlation coefficient matters. *EASE*, 2020, pp. 120–129. DOI: 10.1145/3383219.3383232.

[12] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge. Why don't software developers use static analysis tools to find bugs? *ICSE*, 2013, pp. 672–681. DOI: 10.1109/ICSE.2013.6606613.

[13] M. Christakis and C. Bird. What developers want and need from program analysis: an empirical study. *ASE*, 2016, pp. 332–343.

*Technical report. Corrections and reproduction attempts are welcome via the repository issue tracker.*